

Лабораторная работа №12. Делегирование, lazy, observer-паттерн и сохранение состояния

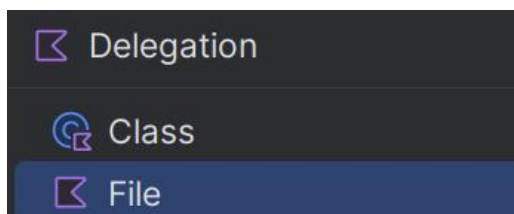
Цель работы. Научиться:

- использовать делегирование;
- применять **lazy**-инициализацию;
- отслеживать изменения состояния через **observable**;
- сохранять и загружать состояние приложения в файл;
- понимать, как эти механизмы применяются в реальных приложениях (в т.ч. Android).

Шаг 1. Делегирование

Делегирование представляет паттерн объектно-ориентированного программирования, который позволяет одному объекту делегировать/перенаправить все запросы другому объекту. В определённой степени делегирование может выступать как альтернатива наследованию. И преимуществом Kotlin в данном случае заключается в том, что Kotlin нативно поддерживает данный паттерн, предоставляя необходимый инструментарий. В Kotlin - это языковая фича (не просто паттерн).

1. Создайте в папке **example** новый файл **Delegation.kt**:



2. Напишем в нём следующий код:

```
interface Base {  
    fun someFun()  
}  
  
class BaseImpl() : Base {  
    override fun someFun() {}  
}  
  
class Derived(someBase: Base) : Base by someBase
```

Есть некоторый интерфейс - **Base**, который определяет некоторый функционал. Есть его реализация в виде класса **BaseImpl**.

И есть ещё один класс - **Derived**, который также применяет интерфейс **Base**. Причем после указания применяемого интерфейса идёт ключевое слово **by**, а после него - объект, которому будут делегироваться вызовы.

То есть в данной схеме класс **Derived** будет делегировать вызовы объекту **someBase**, который представляет интерфейс **Base** и передается через первичный конструктор. При этом **Derived** может не реализовать интерфейс **Base** или реализовать неполностью - какие-то отдельные свойства и функции.

3. Например, рассмотрим следующие классы:

```
interface Messenger{
    fun send(message: String)
}
class InstantMessenger(val programName: String) : Messenger{
    override fun send(message: String){
        println("Message ` $message ` has been sent")
    }
}
class SmartPhone(val name: String, m: Messenger): Messenger by m
```

Здесь определён интерфейс **Messenger**, который представляет условно программу для отправки сообщений. Для имитации отправки сообщений определена функция **send()**.

Также есть класс **InstantMessenger** - программа мгновенных сообщений или проще говоря мессенджер, который применяет интерфейс **Messenger**, реализуя его функцию **send()**.

Далее определен класс **SmartPhone**, который представляет смартфон и также применяет интерфейс **Messenger**, но не реализует его. Вместо этого он принимает через первичный конструктор объект **Messenger** и делегирует ему обращение к функции **send()**. Применим классы:

```
fun main() {
    val max = InstantMessenger(programName = "MAX")
    val yotaPhone = SmartPhone(name = "YotaPhone ", m = max)
    yotaPhone.send("Hello Kotlin")
    yotaPhone.send("Learn delegation")
}
```

Здесь создан объект **yotaPhone**, который представляет класс **SmartPhone**. Поскольку **SmartPhone** применяет интерфейс **Messenger**, то мы можем вызвать у объекта **yotaPhone** функцию **send()** для отправки условного сообщения. Однако сам класс **SmartPhone** НЕ реализует функцию **send()** - само выполнение этой функции делегируется объекту **max**, который в реальности выполняет отправку сообщения. Соответственно при выполнении программы мы увидим следующий консольный вывод:

```
Message `Hello Kotlin` has been sent
Message `Learn delegation` has been sent
```

Множественное делегирование

Подобным образом один объект может делегировать выполнение различных функций разным объектам. Например, добавим новый интерфейс **PhotoDevice** и класс **PhotoCamera**:

```
interface PhotoDevice{
    fun takePhoto()
}

class PhotoCamera: PhotoDevice{
    override fun takePhoto() = println("Take a photo")
}
```

Внесём изменения в наш делегат:

```
class SmartPhone(val name: String, m: Messenger, p: PhotoDevice) :
    Messenger by m, PhotoDevice by p
```

И добавим правки в **main()**:

```
fun main() {
    val max = InstantMessenger(programName = "MAX")
    + val photoCamera = PhotoCamera()
    val yotaPhone = SmartPhone(name = "YotaPhone ", m = max, p = photoCamera)
    yotaPhone.send("Hello Kotlin")
    yotaPhone.send("Learn delegation")
    + yotaPhone.takePhoto()
}
```

При выполнении программы мы увидим следующий консольный вывод:

```
Message `Hello Kotlin` has been sent
Message `Learn delegation` has been sent
Take a photo
```

Здесь класс **SmartPhone** также реализует интерфейс **PhotoDevice**, который предоставляет функцию **takePhoto()** для съемки фото. Но выполнение этой функции он делегирует параметру **p**, который представляет интерфейс **PhotoDevice** и в роли которого выступает объект **PhotoCamera**.

Переопределение функций

Класс может переопределять часть функций интерфейса, в этом случае выполнение этих функций не делегируется. Например внесём изменения в интерфейс **Messenger** и класс **InstantMessenger**:

```
interface Messenger {
    fun sendTextMessage()
    fun sendVideoMessage()
}

class InstantMessenger(val programName: String) : Messenger {
    override fun sendTextMessage() = println("Send text message")
    override fun sendVideoMessage() = println("Send video message")
}
```

Изменяем класс **SmartPhone**:

```
class SmartPhone(val name: String, m: Messenger) : Messenger by m {
    override fun sendTextMessage() = println("Send sms")
}
```

И вносим изменения в **main()**:

```
fun main() {
    val max = InstantMessenger(programName = "MAX")
    //val photoCamera = PhotoCamera()
    val yotaPhone = SmartPhone(name = "YotaPhone ", m = max)
    yotaPhone.sendTextMessage()
    yotaPhone.sendVideoMessage()
}
```

В данном случае класс **SmartPhone** реализует функцию **sendTextMessage()**, поэтому ее выполнение не делегируется. Консольный вывод программы:

```
Send sms  
Send video message
```

Делегирование свойств

В **Kotlin** делегирование свойств — это отдельная мощная фишка, которая отличается от делегирования интерфейсов. Она позволяет вынести логику получения и установки значения свойства в отдельный объект, который называется делегатом.

Импортируем в наш файл пакет для работы с делегатами:

```
import kotlin.properties.Delegates
```

Добавим пример делегирования свойств:

```
var counter: Int by Delegates.observable(initialValue = 0) { _, old, new ->  
    println("Счётчик изменился: $old -> $new")  
}
```

В **main()** добавим:

```
fun main() {  
    counter = 1  
    counter = 5  
}
```

Что происходит:

- Свойство **counter** делегировано объекту, возвращаемому **Delegates.observable(0)**.
- При каждом присваивании (**counter = ...**) автоматически вызывается лямбда, которая выводит старое и новое значение.
- Это позволяет реактивно отслеживать изменения свойства без ручного написания геттера/сеттера.

Вывод программы:

```
Счётчик изменился: 0 -> 1  
Счётчик изменился: 1 -> 5
```

Это — делегирование свойств, и оно работает даже для свойств в классах, например:

```
class User {
    var name: String by Delegates.observable( initialValue = "<no name>" ) { _,
        old, new ->
            println("Имя изменено: '$old' → '$new'")
        }
}
```

После внесения изменений выполните команды:

git add .

git commit -m "add delegation example"

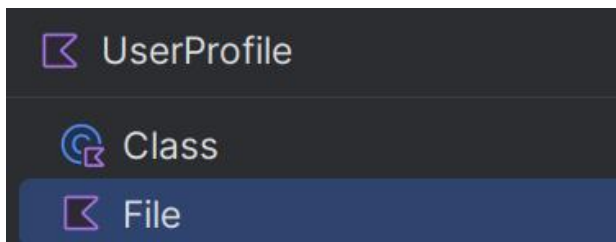
git push

Шаг 2. Система управления профилем пользователя в приложении

Задача: Мы хотим создать простую систему управления профилем пользователя в приложении. Профиль должен:

- хранить **имя** и **email**;
- загружать аватар **только при первом обращении** (ленивая инициализация);
- **отслеживать изменения email** и выводить уведомление в консоль;
- использовать **делегирование свойств**, чтобы не писать лишний шаблонный код.

Для этого создадим новый файл в папке example под названием **UserProfile.kt**.



Реализация

Откроем файл **UserProfile.kt** и напишем следующий код:

```
import kotlin.properties.Delegates

class UserProfile(initialName: String, initialEmail: String) {}
```

Внутри класса **UserProfile** определяем:

1. **Имя** — обычное изменяемое свойство, но с делегированием через observable

```
var name: String by Delegates.observable(initialValue = initialName) {
    _, old, new ->
        println("Имя изменено: '$old' → '$new'")
}
```

2. **Email** — тоже отслеживаем изменения

```
var email: String by Delegates.observable(initialValue = initialEmail) {
    _, old, new ->
        println("Email обновлён: '$old' → '$new'")
}
```

3. **Аватар** — загружается лениво, только при первом обращении

```
val avatar: String by lazy {
    println("Загружаем аватар для $name...")
    "avatar_of_$name.png"
}
```

Теперь добавим функцию **main()**, чтобы протестировать поведение:

```
fun main() {
    println("Создаём профиль пользователя...")
    val user = UserProfile(initialName = "Алиса", initialEmail = "alice@example.com")

    println("\nИмя: ${user.name}")
    println("Email: ${user.email}")

    println("\nОбращаемся к аватару впервые:")
    println("Файл аватара: ${user.avatar}")

    println("\nОбращаемся к аватару снова (должен быть взят из кэша):")
    println("Файл аватара: ${user.avatar}")

    println("\nМеняем email:")
    user.email = "alice_new@example.org"

    println("\nМеняем имя:")
    user.name = "Алиса К."
}
```

Что мы использовали:

- **by Delegates.observable(...)** — делегирует геттер и сеттер свойства, позволяя выполнять код при каждом изменении значения. Идеально для логирования или реакции на изменения.
- **by lazy { ... }** — обеспечивает **ленивую инициализацию**: блок кода выполняется **ровно один раз** — при первом обращении к свойству. Полезно для ресурсоёмких операций (загрузка данных, файлов, подключение к БД и т.д.).
- Оба механизма — это примеры **делегирования свойств** через ключевое слово **by**.

Вывод программы:

```
Создаём профиль пользователя...

Имя: Алиса
Email: alice@example.com

Обращаемся к аватару впервые:
Загружаем аватар для Алиса...
Файл аватара: avatar_of_Алиса.png

Обращаемся к аватару снова (должен быть взят из кэша):
Файл аватара: avatar_of_Алиса.png

Меняем email:
Email обновлён: 'alice@example.com' → 'alice_new@example.org'

Меняем имя:
Имя изменено: 'Алиса' → 'Алиса К.'
```

Мы компактно и безопасно реализовали реактивное поведение и ленивую загрузку без ручного написания геттеров/сеттеров, используя встроенные механизмы Kotlin. Это делает код чище, короче и выразительнее.

После внесения изменений выполните команды:

git add .

git commit -m "add delegation user profile"

git push

Шаг 3. Игровой персонаж

На этот раз мы создадим игрового персонажа, который получает способности с помощью делегирования свойств.

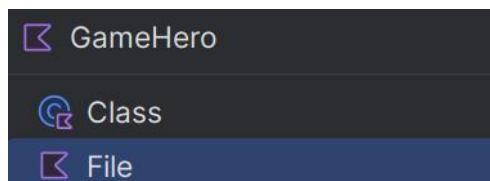
Задача и идея

Задача: Представим, что мы делаем простую RPG-игру. У персонажа есть:

- **энергия (mana)** — обновляется со временем, но мы пока просто будем её отслеживать;
- **особая способность (ultimate)** — активируется только один раз за игру, и её инициализация «дорогая» (например, загрузка анимации или звука);
- **имя** — может меняться, и при каждом изменении мы хотим выводить сообщение в лог (например, для чит-кода или переименования в игре).

Чтобы не писать громоздкие сеттеры и оптимизировать загрузку, воспользуемся делегированием свойств в Kotlin.

Создадим файл **GameHero.kt** в папке **example**:



Реализация

В файле **GameHero.kt** напишем:

```
import kotlin.properties.Delegates

class UserProfile(initialName: String, initialEmail: String) {}
```

Внутри класса **GameHero** определяем:

```
import kotlin.properties.Delegates

class GameHero(startName: String) {}
```

1. Имя персонажа — отслеживаем изменения

```
var name: String by Delegates.observable(initialValue = startName) {
    _, old, new ->
    println("[old] получил новое имя: $new!")
}
```

2. Энергия (мана) — тоже отслеживаем

```
var mana: Int by Delegates.observable(initialValue = 100) {
    _, old, new ->
    println("Мана: $old -> $new")
}
```

3. Супер-способность — ленивая инициализация: загружаем "только при первом использовании"

```
val ultimate: String by lazy {  
    println("Загружаем анимацию способности...")  
    "Метеоритный дождь"  
}
```

Теперь добавим функцию **main()**, чтобы протестировать поведение:

```
fun main() {  
    println("Создаём героя...")  
    val hero = GameHero( startName = "Воин Света")  
  
    println("\nТекущая мана: ${hero.mana}")  
  
    println("\nГерой пытается использовать способность:")  
    println("Способность: ${hero.ultimate}")  
  
    println("\nПовторный вызов способности (должна быть мгновенной):")  
    println("Способность: ${hero.ultimate}")  
  
    println("\nМеняем имя героя:")  
    hero.name = "Тёмный Паладин"  
  
    println("\nГерой восстанавливает ману:")  
    hero.mana = 200  
}
```

Что мы использовали:

- **Delegates.observable** — чтобы **реагировать на изменения имени и маны** (полезно для логов, анимаций, обновления интерфейса).
- **by lazy** — чтобы **загрузить "тяжёлую" способность только один раз**, при первом обращении (экономия памяти и времени запуска).
- Всё это — **делегирование свойств**, которое делает код чище и выразительнее.

Вывод программы:

```
Создаём героя...

Текущая мана: 100

Герой пытается использовать способность:
Загружаем анимацию способности...
Способность: Метеоритный дождь

Повторный вызов способности (должна быть мгновенной):
Способность: Метеоритный дождь

Меняем имя героя:
[Воин Света] получил новое имя: Тёмный Паладин!

Герой восстанавливает ману:
Мана: 100 → 200
```

Почему это круто в играх?

- **Ленивая загрузка** — не грузим все анимации и звуки при старте, а только когда они нужны.
- **Отслеживание изменений** — можно легко подключить обновление HUD, логику квестов или сетевую синхронизацию.
- **Чистый код** — никаких ручных setName(), onManaChanged() и т.п.

После внесения изменений выполните команды:

git add .

git commit -m "add delegation game hero"

git push

Шаг 4. Отслеживание изменений ресурсов (observable)

Добавим наблюдение за изменением количества ресурса.

Отслеживание изменений ресурсов (observable)

Обновляем **OutpostResource**.

1. Добавляем импорт для работы с делегатами (Delegates):

```
import kotlin.properties.Delegates
```

2. Изменяем параметр **amount** на **amountInit** (делаем его неизменяемым):

```
data class OutpostResource(  
    val id: Int,  
    val name: String,  
    + val amountInit: Int  
) {
```

3. Добавляем делегированное свойство с **observable**:

```
) {  
    var amount: Int by Delegates.observable(initialValue = amountInit) { _, old, new ->  
        println("Ресурс [$name] изменился: $old -> $new")  
    }  
}
```

- by Delegates.observable() — это делегированное свойство
- Автоматически отслеживает изменения при каждом присваивании
- Вызывает обработчик (лямбда-функцию) при каждом изменении

Параметры лямбды:

- ◆ _ — игнорируемое свойство (нам не нужно)
- ◆ old — старое значение
- ◆ new — новое значение

Тут же в методе **main()** измените параметр, чтобы исправить ошибку:

```
fun main() {  
    val gas = OutpostResource(id = 1, name = "Gas", amountInit = 100)  
    val mineral = OutpostResource(id = 2, name = "Minerals", amountInit = 250)  
    println("Успех! Вы добыли дополнительное количество минералов: ${mineral.amount + 50}")  
    val bonusMineral = mineral.copy(id = 3, name = "Minerals Bonus", amountInit = mineral.amount + 50)  
    println(gas.toString())  
    println(mineral.toString())  
    println(bonusMineral.toString())  
}
```

Lazy-инициализация системы логирования

Добавим ленивую инициализацию тяжёлого объекта. В **Main.kt** добавить внутрь файла:

```
object SystemLogger {  
    init {  
        println("SystemLogger инициализирован")  
    }  
  
    fun log(message: String) {  
        println("[LOG] $message")  
    }  
}
```

Используем **lazy** в **Main.kt**:

```
val logger by lazy {  
    SystemLogger  
}
```

Вызовем в методе **main()**:

```
fun main() {  
    + logger.log("Запуск базы")  
}
```

- **SystemLogger** создаётся только при первом обращении;
- если логгер не нужен — он не создаётся;
- полностью **потокобезопасно**.

Сохранение состояния базы в файл

Любая реальная система должна:

- сохранять своё состояние;
- восстанавливаться после перезапуска.

Создаём в корне проекта файл с именем **FileStorage.kt**

```
@FileStorage
```

В нём добавляем импорты:

```
import resources.OutpostResource  
import java.io.File
```

- *Импортируем наш data class с ресурсами*
- *Импортируем класс File для работы с файлами*

Создаём **object FileStorage**

```
object FileStorage {}
```

Зачем **object**, а не **class**:

- Это **синглтон** — будет только один экземпляр на всё приложение
- Не нужно создавать объект через **FileStorage()**
- Доступен отовсюду как **FileStorage.save()**

Объявляем константу для имени файла

```
object FileStorage {  
    + private const val FILE_NAME = "outpost_state.txt"
```

- **private** — чтобы никто извне не мог изменить имя файла
- **const val** — константа времени компиляции

- Храним имя файла в одном месте для удобства изменения

Создаём функцию save()

```
fun save(resources: List<OutpostResource>) {  
  
}
```

- Принимает список ресурсов для сохранения
- Сохраняет их в файл в удобном формате

Внутри создадим объект файла:

```
fun save(resources: List<OutpostResource>) {  
    +val file = File(pathname = FILE_NAME)
```

- **File** — класс Java/Kotlin для работы с файловой системой
- **FILE_NAME** — используем нашу константу

Далее запишем текст в файл:

```
file.writeText(  
    text = resources.joinToString(separator = "\n") {  
        "${it.id.toString().padEnd(length = 3)} | ${it.name.padEnd(  
            length = 10)} | ${it.amount}"  
    }  
)  
println("Состояние базы сохранено в файл")
```

Что делает joinToString:

1. Берёт список ресурсов
2. Для каждого ресурса вызывает лямбду, которая форматирует строку:
 - **it.id.toString().padEnd(3)** — ID выравниваем до 3 символов
 - **it.name.padEnd(10)** — имя выравниваем до 10 символов
 - **it.amount** — количество

Создаём функцию load()

```
fun load(): List<OutpostResource> {  
  
}
```

- Возвращает список ресурсов

Далее создаём объект того же файла, если его нет возвращаем пустой список:

```
fun load(): List<OutpostResource> {  
    + [ val file = File(pathname = FILE_NAME)  
      if (!file.exists()) return emptyList()  
    ]  
}
```

Проверка file.exists():

- Предотвращает ошибку при попытке чтения несуществующего файла
- Возвращает пустой список — база пустая

Информируем пользователя:

```
println("Загрузка состояния базы из файла")
```

Далее читаем все строки файла как List<String>, преобразуем каждую строку в OutpostResource. Разделяем строку по точке с запятой и создаём объект:

```
return file.readlines().map {  
    val (id, name, amount) = it.split(...delimiters = ";")  
    OutpostResource(id.toInt(), name, amountInit = amount.toInt())  
}
```

Деструктуризация val (id, name, amount):

- Автоматически присваивает значения из списка переменным
- **line.split(";")** разбивает строку "1;Дерево;100" на ["1", "Дерево", "100"]
- **id.toInt()** и **amount.toInt()** — преобразуем строки в числа

Что мы создали:

1. Простое файловое хранилище для нашей игры/приложения
2. Синглтон — доступен отовсюду без создания экземпляра
3. Функции для сохранения/загрузки состояния игры
4. Автоматическое форматирование данных для удобного чтения
5. Обработку ошибок (файл не существует)

Интеграция сохранения в ResourceManager

Обновим **ResourceManager**:

```
fun getAll(): List<OutpostResource> = resources.toList()
```

Финальный запуск (Main.kt)

В методе **main()** добавим загрузку состояния:

```
// Загрузка состояния
val loadedResources = FileStorage.load()
loadedResources.forEach { manager.add(it) }
```

Добавляем ресурсы:

```
if (loadedResources.isEmpty()) {
    manager.add(OutpostResource(id = 1, name = "Minerals", amountInit = 300))
    manager.add(OutpostResource(id = 2, name = "Gas", amountInit = 100))
}
```

В самом конце метода **main()** сохраним состояние в файл:

```
FileStorage.save(resources = manager.getAll())
```

Сохранение изменений в системе контроля версий

После завершения работы с выполните следующие команды в терминале:

git add .

git commit -m "final: delegation, lazy and file persistence"

git push

Шаг 5. Добавление описания в README.md

В файле **README.md** добавьте описание изученных тем и примеры их применения в Kotlin.

Темы:

- Делегирование свойств
- lazy
- Observer-паттерн (наблюдатель)

Примерный текст для вставки (можете добавить свои примеры):

Делегирование свойств

Делегирование свойств позволяет передать логику хранения и обработки значения другому объекту.

В Kotlin это реализуется с помощью ключевого слова **by**.

Преимущества:

уменьшение дублирования кода;

централизованная логика проверки и обработки данных;

более чистый и читаемый код.

Пример: ограничение диапазона значения энергии

```
var energy: Int by Delegates.observable(100) { _, old, new ->
    println("Энергия изменилась: $old → $new")
}
```

Lazy (ленивая инициализация)

lazy позволяет инициализировать объект только при первом обращении к нему.

Это полезно, если:

объект создаётся не всегда;

его создание ресурсоёмкое;

нужно отложить инициализацию.

Пример:

```
val resourceManager by lazy {  
    ResourceManager()  
}
```

Объект `ResourceManager` будет создан только при первом использовании.

Observer-паттерн (наблюдатель)

Observer-паттерн позволяет объектам реагировать на изменения состояния другого объекта.

В проекте `Galaxy Outpost Manager` наблюдатели могут:

реагировать на изменение ресурсов;

логировать события;

уведомлять пользователя.

Пример идеи:

`ResourceManager` изменяет ресурсы;

наблюдатель выводит сообщение в консоль при изменении.

Сохранение состояния

Для сохранения состояния проекта используется сериализация в `JSON`.

Это позволяет:

сохранять данные между запусками программы;

хранить состояние в человекочитаемом формате;

легко перенести логику в `Android`-приложение.

После внесения изменений выполните команды:

`git add .`

`git commit -m "docs: add delegation, lazy, observer and state saving description"`

`git push`

Самостоятельное задание

Тема: Система состояния базы аванпоста

Структура проекта

Все файлы должны находиться в одном пакете:

`outpostState`

Шаг 1. Делегирование свойства ресурса

1. Создайте файл **`ObservableResource.kt`**.

2. Реализуйте класс ресурса, который:

- хранит имя ресурса;
- хранит количество ресурса;
- использует `Delegates.observable` для отслеживания изменения количества.

3. При изменении количества ресурса выводите сообщение в консоль.

Пример сообщений:

- «Ресурс `Minerals` изменён: 100 → 120»

Шаг 2. Lazy-инициализация менеджера

1. Создайте файл **OutpostManager.kt**.
2. Реализуйте менеджер, который:
 - содержит список ресурсов;
 - инициализируется с помощью `by lazy`.
3. Убедитесь, что сообщение о создании менеджера выводится только при первом обращении.

Шаг 3. Observer-подписчик

1. Создайте файл **ResourceObserver.kt**.
2. Реализуйте простой наблюдатель, который:
 - реагирует на изменение ресурсов;
 - выводит информацию в консоль.
3. Подключите наблюдатель к ресурсу.

Шаг 4. Сохранение состояния

1. Создайте файл **StateStorage.kt**.
2. Реализуйте:
 - сохранение списка ресурсов в файл;
 - загрузку ресурсов из файла.

Шаг 5. Точка входа

1. Создайте файл **Main.kt**.
2. В **main()**:
 - создайте менеджер;
 - добавьте несколько ресурсов;
 - измените их количество;
 - сохраните состояние;
 - загрузите состояние и выведите результат в консоль.

Шаг 6. Контроль версий

После выполнения задания выполните команды:

git add .

git commit -m "add delegation, lazy, observer and state saving practice"

git push